Lista Encadeada Circular
Lista Duplamente Encadeada

Relembrando listas encadeadas

- Um nó em uma Lista Encadeada possui basicamente dois itens:
  - ponteiro para o próximo
  - informação armazenada

- Caso a informação não seja um dado simples:
  - ➔criar vários campos, um para cada informação

Ex.: código, preço e quantidade de um produto

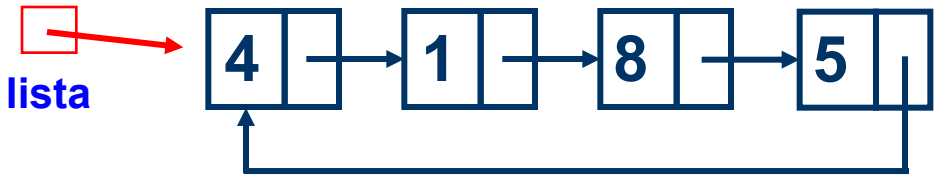
Relembrando listas encadeadas

```
typedef struct tp_no {
    int cod;
    float preco;
    int quant;
    struct tp_no *prox;
} TPLISTA;

TPLISTA *lista;
...
lista->cod=1;
lista->preco=10.5;
lista->quant=20;
```

Listas Circulares

- O último elemento tem como próximo o primeiro elemento da lista, formando um ciclo
- Útil quando:
  - a busca é feita a partir de qualquer elemento
  - não há ordenação na lista
- A rigor não existe "primeiro" ou "último"
- Ainda é necessário que exista um ponteiro para algum elemento, para a localização da lista
  - por convenção, referência do primeiro ou do último



Listagem

```
void listagem (tplista *t) {
    tplista *p=t;
    if (t!=NULL)
        do {
            printf("Info: %d", p->info);
            p=p->prox;
        } while (p!=t);
    else
        printf("Lista Circular vazia!");
}
```

Lista Circular

Como seria um procedimento de inserção em uma Lista Circular?

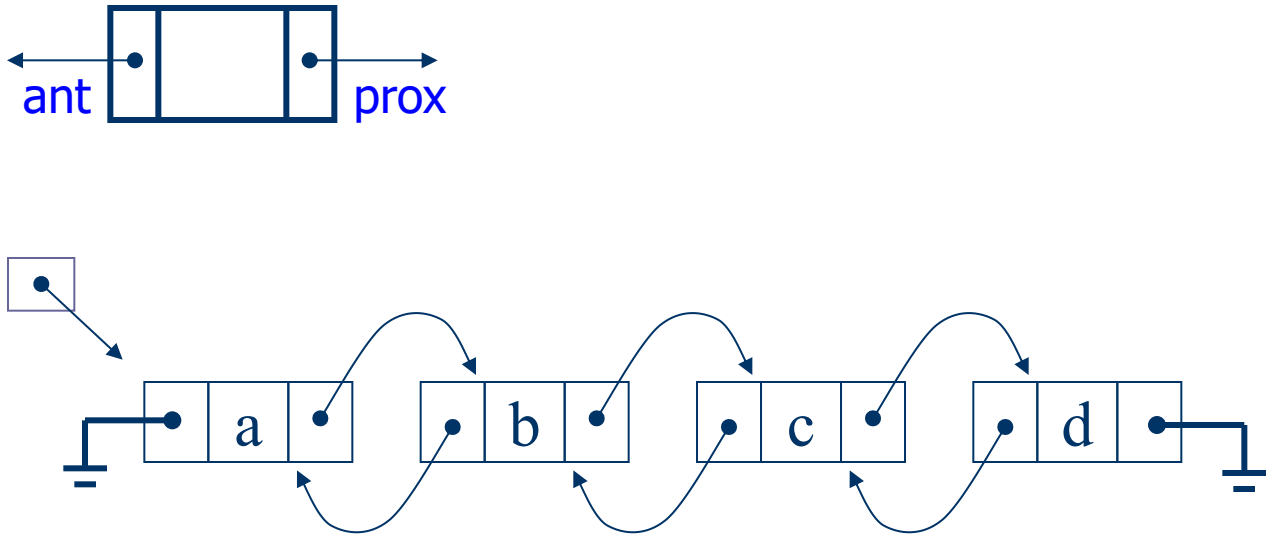
E um procedimento de remoção?

Lista Duplamente Encadeada

- Útil quando é preciso percorrer a lista na ordem inversa
- Remoção de um elemento não precisa guardar anterior
- Remoção de um elemento cujo ponteiro é informado não precisar percorrer a fila toda
- Um conjunto maior de ligações precisam ser atualizadas

Lista Duplamente Encadeada

- Cada nó possui dois ponteiros: um para o elemento anterior e outro para o próximo elemento (ant e prox)



Listas Duplamente Encadeada

```
typedef int tpitem;

typedef struct tp_no {
   tpitem info;
   struct tp_no *ant;
   struct tp_no *prox;
} tplista;

tplista *lista;
```

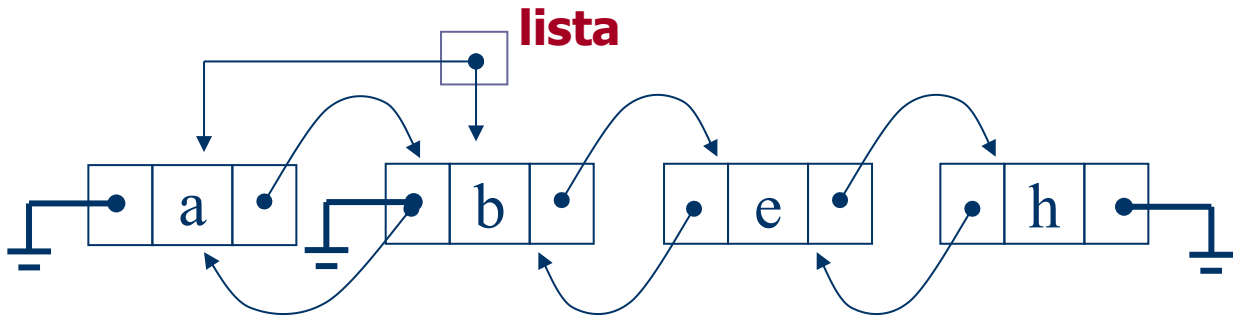
Busca e Listagem

- Busca e Listagem:

Código igual ao que é utilizado para a Lista Simplesmente Encadeada

Inserção no Início

- O novo elemento é encadeado no início da lista
- O seu próximo passa a ser o antigo primeiro elemento e o seu anterior é NULL
  - Se a lista não estiver vazia, o anterior do o antigo primeiro passa a ser o novo elemento
- O ponteiro da lista é passado por referência e atualizado para apontar para o novo nó
- A função retornar 1 ou zero indicando o sucesso da inclusão



Inserção

```
int insere(tplista **t, tpitem e){
    tplista *novo;
    novo = aloca();
    if (!novo)
        return 0;
    novo->info = e;
    novo -> ant = NULL;
    novo -> prox = *t;
    if ((*t) != NULL)
        (*t) -> ant = novo;
    *t = novo;
    return 1;
}
```

Remoção

- A remoção é mais trabalhosa, pois é preciso acertar a cadeia nos dois sentidos
- Em compensação, pode-se retirar um elemento conhecendo-se apenas o ponteiro para ele
- Utiliza-se uma função de busca para localizar o elemento e em seguida o encadeamento é ajustado
- Ao final, o elemento é liberado

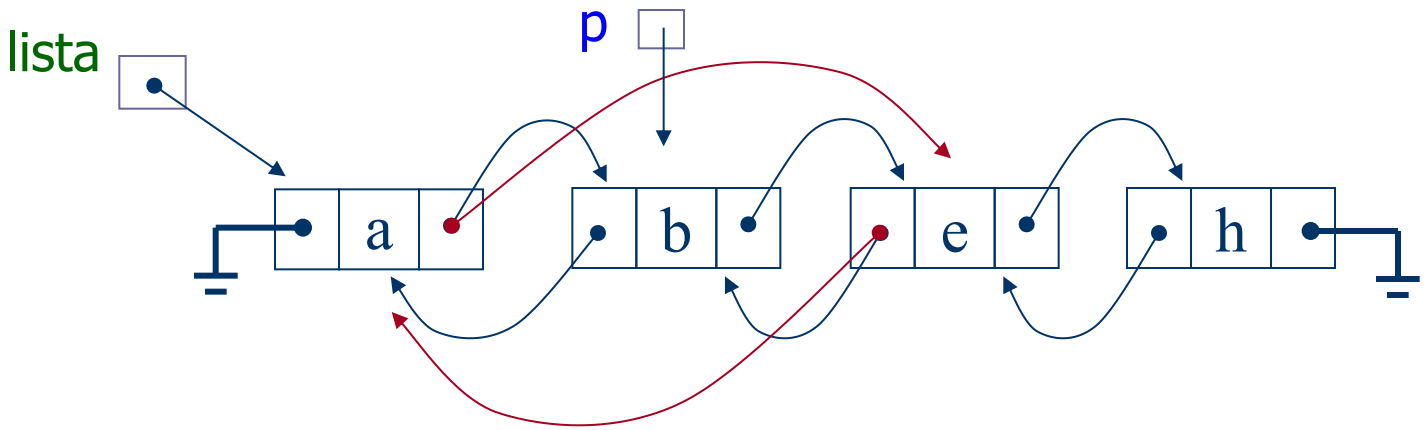
Remoção

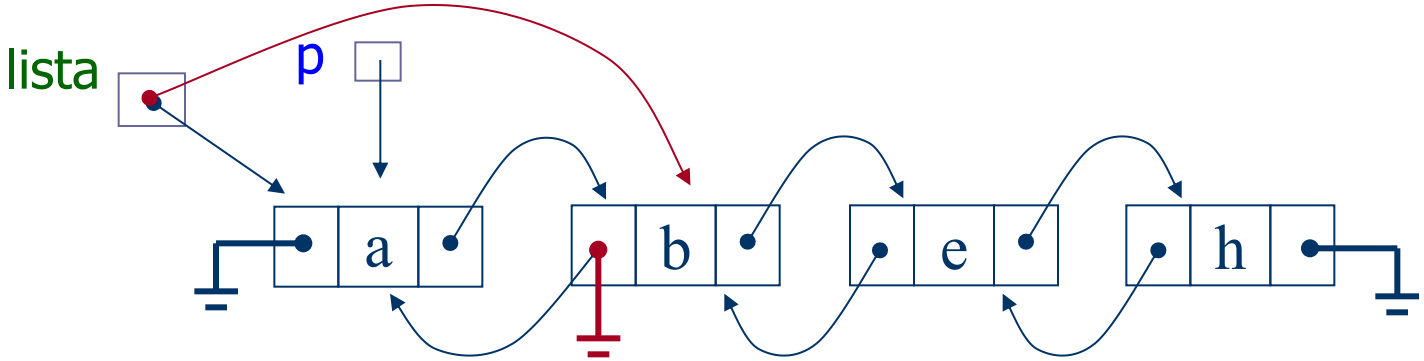
- Sendo **p** o ponteiro para o elemento a ser excluído, se o elemento estiver no meio da lista, devemos fazer:

```
p->ant->prox = p->prox;
p->prox->ant = p->ant;
```

- Caso o elemento esteja em um extremo da lista, existem outras condições:
  - se p for o primeiro, não se pode referenciar p->ant, pois ele é NULL; o mesmo acontece para p->prox quando é o último
  - além disso, se for o primeiro, é preciso atualizar o ponteiro da lista

Remoção



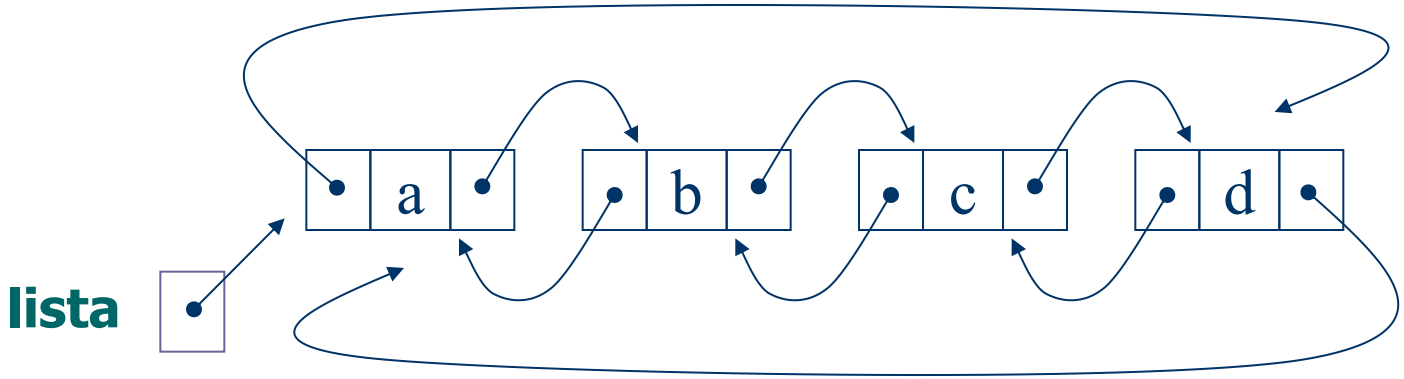


Remoção

```
int retira (tplista **t, int valor) {
    tplista *p;
    p=busca(*t,valor);
    if (p == NULL)
        return 0;                    /* não achou o elemento */
    if (*t == p)
        *t=p->prox;                  /* primeiro elemento */
    else
        p->ant->prox=p->prox;    /* não é o primeiro */
    if (p->prox!=NULL)               /* não é o último */
        p->prox->ant=p->ant;
    free(p);
    return 1;
}
```

Lista Duplamente Encadeada e Circular

- Cada nó possui dois ponteiros: um para o elemento anterior e outro para o próximo elemento (ant e prox)
- O anterior do primeiro é o último e o próximo do último é o primeiro



Praticar!